ITEC 102 Final Project

User manual

A C# Console Application for Local Password Management and Password Generation

Prepared by	Winslie Ghlen A. Aragones Jared A. Manito Khen Joshki D. Rafael
Application type	C# console application
Storage	Local file: accounts.txt

This manual describes the menu options, input rules, local data file, and security limitation of the submitted program.

1. Before you begin

Open the C# project in Visual Studio or another environment that supports the project's .NET version. Build the project, then run the console application. The application is operated entirely through typed menu choices and responses.

Important security notice

The current program saves platform names, usernames, and passwords to accounts.txt as plain text. The file is not encrypted. Use test accounts and test passwords only. Do not save banking, school, work, or other sensitive credentials in this project.

When the program starts

The program checks for the saved accounts file. If it exists, saved records are loaded. If it does not exist, the program starts with an empty account list. After each action, the program returns to the main menu.

Main menu

Choice	Action
1	Add Account
2	View All Accounts
3	Search Account
4	Update Account
5	Delete Account
6	Generate Random Password
7	Exit and save

2. Managing accounts

Add Account

Choose **1**. Enter a platform name and username. Empty values are rejected. For the password, choose whether to enter your own value or generate a random password. The completed record is added to the account list.

View All Accounts

Choose **2** to display saved records in a numbered list showing each platform, username, and password. If there are no records, the program reports that there are no saved accounts.

Search Account

Choose **3** and enter a platform keyword. The search ignores uppercase and lowercase differences. Matching records are displayed; if none match, the program reports that no matching account was found.

Update Account

Choose **4**, select an account number, and enter a replacement username and password. A non-empty replacement updates that field; an empty replacement keeps the existing field. Invalid or out-of-range numbers are rejected.

Delete Account

Choose **5**, select an account number, and answer the confirmation prompt with **Y** or **N**. **Y** removes the record. **N** cancels the deletion.

3. Generating a password

Choose **6** from the main menu, or choose the generate option while adding an account. The program asks for a length from 8 to 32 characters, then asks whether to include letters, numbers, and symbols.

Steps

• Enter a length between 8 and 32.

• Answer Y or N for letters, numbers, and symbols.

• Read the generated password printed by the program.

If all three character types are declined, the program automatically uses letters. Invalid lengths and invalid Y/N answers are rejected and requested again.

4. Saving and closing

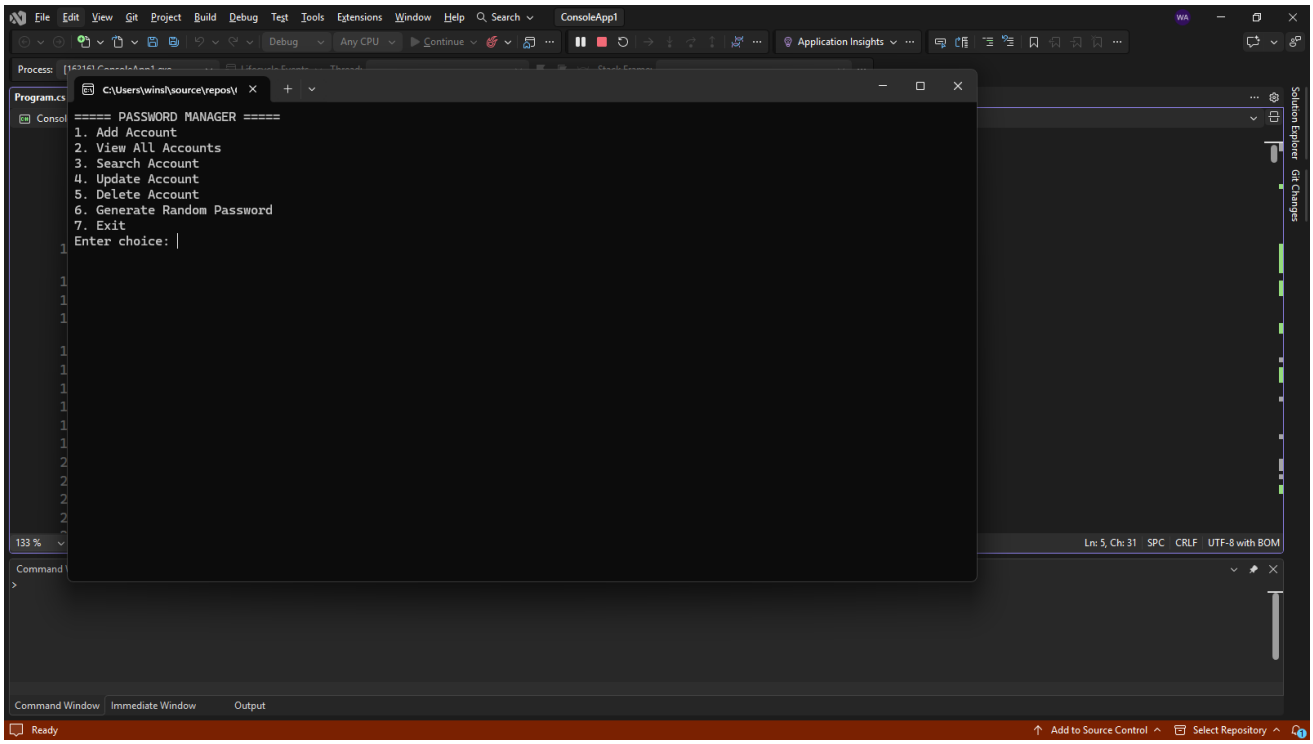
Choose **7. Exit** when finished. The program writes the current account list to **accounts.txt**, displays a goodbye message, and stops. If the program is closed before Exit is completed, the latest changes may not be written.

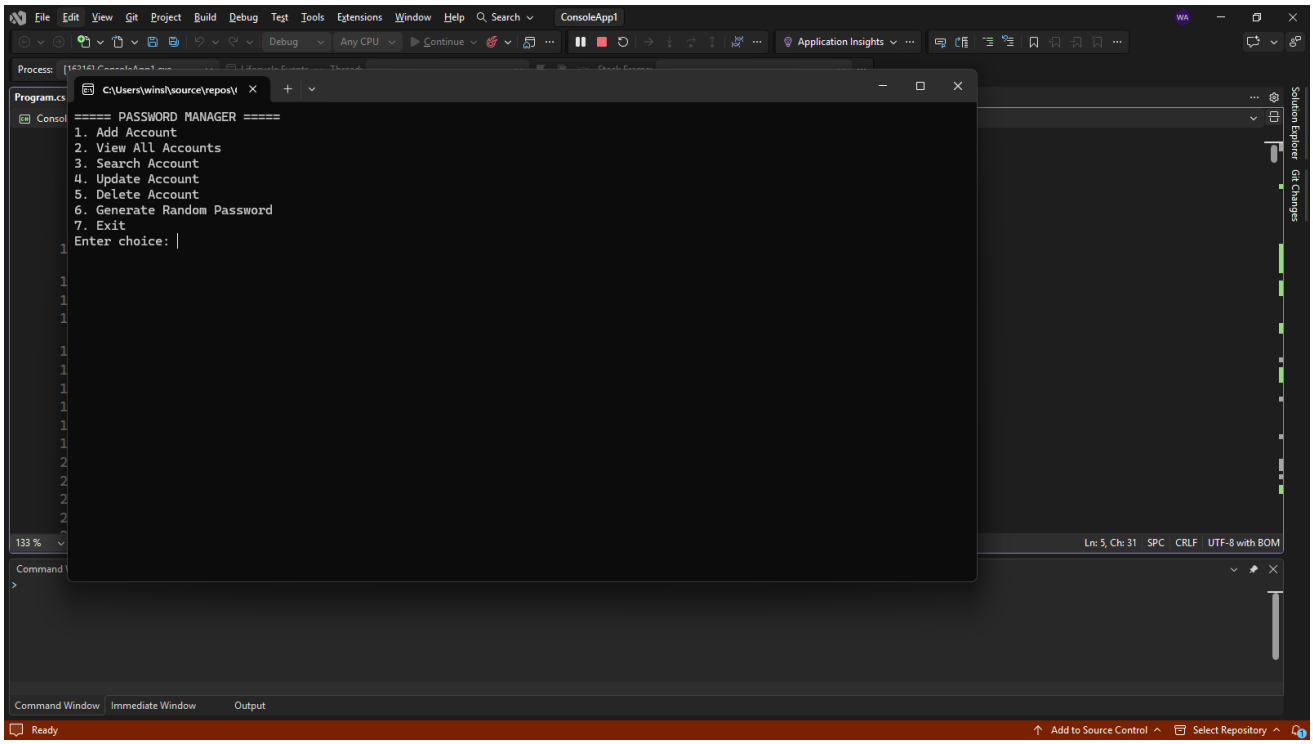
Working with accounts.txt

The file is local to the program's working directory. Keep it with the program if you want the same records to load next time. Because the contents are plain text, protect the computer and the file itself. Do not upload the file with real credentials to a public repository.

5. Program screenshots

The submitted screenshots show the application running from Visual Studio with the console menu visible. The menu includes the seven operations documented in this manual.





The screenshots show the starting menu; the prompts and results for each action appear as the corresponding menu option is used.

6. Scope and limitations

This is a local, single-user console application. It does not include a graphical, web, or mobile interface; cloud synchronization; multi-device access; browser autofill; biometric login; two-factor authentication; enterprise-grade encryption; or import and export with another password manager. It demonstrates basic C# variables, methods, validation, loops, file handling, and console input/output.